to *'cout'* in traditional C++ programming.

So how does one compile it? The Trolltech people came out with an easy solution to support cross-platform compilation. First, a project file should be created, and then a *Makefile* is created using it. Then, just run *make* to compile the program. When you install QT Creator, a utility called *'qmake'* is also installed. This is a cross-platform *Makefile* generator for Qt. Check out its *man* pages for more. Run *qmake -project* to create a project file (*.pro* extension, with its name that of the containing directory, i.e., *first*).

```
$ cat first.pro
#############################################################
####
# Automatically generated by qmake (2.01a) Mon Nov 28 05:49:29 2011
#############################################################
####

TEMPLATE = app
TARGET =
DEPENDPATH += .
INCLUDEPATH += .

# Input
SOURCES += main.cpp
```

You can create both applications and libraries; the value of TEMPLATE is *'app'*, indicating this is an application. I will cover library development in later articles. The SOURCES entry lists source files in the project, about which more details appear later in this article series. Now, let us generate the *Makefile* with *qmake*. It's a long file—it may contain up to 200 lines. Run *make* to create the executable file:

```
$ make
g++ -c -pipe -O2 -Wall -W -D_REENTRANT -DQT_NO_DEBUG -DQT_GUI_LIB
-DQT_CORE_LIB -DQT_SHARED -I/usr/share/qt4/mkspecs/linux-g++ -I.
-I/usr/include/qt4/QtCore -I/usr/include/qt4/QtGui -I/usr/include/
qt4 -I. -I. -o main.o main.cpp
make: Circular all <- first dependency dropped.
g++ -Wl,-O1 -o first main.o     -L/usr/lib -lQtGui -lQtCore -lpthread
```

In the last line, you can see that our sample program uses the POSIX thread library too. Run the file with *./first* to see the output *Hello world*. It's done! We have successfully compiled our first program.

Sample GUI program

Qt has an option to create UI files using a drag-and-drop method, which we will explore in the next article. For now, let us hand-code a sample simple GUI program:

```
#include<QApplication>
#include<QLabel>
```

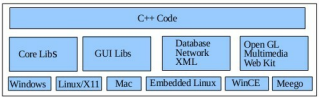

Figure 1: Qt architecture

Figure 2: Widget label

```
int main(int argc, char *argv[]){
      QApplication a(argc, argv);
      QLabel label;
      label.setText("Hello World");
      label.show();
      a.exec();
}
```

In the first example, there was no GUI. The program terminates when *main* returns. However, in GUI programs, we can't do that, or the application will not be usable. We want the GUI to run until the user closes the window. To achieve this, run your program in a loop till this happens, so you use the event-loop-based class *QApplication*. When you create an object of this class and call its *exec()* function, *main* never returns. The application keeps on waiting for user input events.

The second header file contains the class *QLabel*, a simple widget used to display text. Instantiate it and set its text to 'Hello World'. When a widget's *show()* function is called, then it becomes a window. So the widget label will be seen like a window. In Figure 2, you can see the resultant label window with title bar. You can resize the output window simply with the help of the mouse.

In the next articles, we will cover the core classes of Qt. In the meanwhile, I suggest you go through the documentation available at *qt.nokia.com*. END

By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at *manoj@knitsolutions.com* or at his blog *www.open4india.blogspot.com* to share your Linux knowledge.